ECAI 2025
I. Lynce et al. (Eds.)
© 2025 The Authors.
This article is published online with Open Access by IOS Press and distributed under the terms of the Creative Commons Attribution Non-Commercial License 4.0 (CC BY-NC 4.0).
doi:10.3233/FAIA251387

A Hybrid CMSA–Column Generation Approach to Variable-Sized Bin Packing

Mehmet Anıl Akbay[a, b,*,1], **Christian Blum**[a,1] **and Günther R. Raidl**[c]

[a]Artificial Intelligence Research Institute (IIIA-CSIC), Campus of the UAB, Bellaterra, 08193 Spain
[b]Autonomus University of Barcelona
[c]Institute of Logic and Computation, TU Wien, Vienna, Austria
ORCID (Mehmet Anıl Akbay): https://orcid.org/0000-0001-7376-7008, ORCID (Christian Blum): https://orcid.org/0000-0002-1736-3559, ORCID (Günther R. Raidl): https://orcid.org/0000-0002-3293-177X

Abstract. This paper introduces a hybrid algorithm that integrates the Construct, Merge, Solve and Adapt (CMSA) metaheuristic with Column Generation (CG) to solve the Variable-Sized Bin Packing Problem (VSBPP). The VSBPP, which requires packing items of varying sizes into bins of different types and costs, is a core combinatorial optimization problem with applications in logistics and resource allocation. Our approach builds upon a previously proposed CMSA-based framework by incorporating a column generation procedure to strengthen the linear programming relaxation and generate improved patterns. In each iteration, heuristic solution construction and merging phases produce a pool of feasible packing patterns. These are then used in a linear programming master problem, from which dual values are extracted and used to solve a knapsack pricing subproblem. This identifies new patterns (columns) with negative reduced cost to be added to the model. Finally, an integer programming phase refines the solution using a reduced subinstance. The algorithm is evaluated on benchmark instances and compared with both the previously published standalone CMSA approach and a standalone column generation method. Experimental results show that the combined CMSA-COLGEN approach consistently outperforms both baselines in terms of solution quality and robustness, particularly on larger and more complex instances.

1 Introduction

Combinatorial optimization problems [17] have long played a central role in artificial intelligence and operations research, underpinning a multitude of real-world applications. Due to the NP-hard nature of most practically relevant ones, solving them efficiently remains a major challenge. Solution approaches are typically classified into exact and heuristic methods [6]. Despite recent advancements in exact methods, they remain limited in scalability and typically require substantial computational resources, especially for large instances of hard problems. On the other hand, while lacking optimality guarantees, heuristics and metaheuristics typically provide more scalable and time-efficient alternatives. To bridge the gap between quality and efficiency, hybrid approaches that combine exact and heuristic techniques have emerged, offering a promising balance for tackling increasingly complex combinatorial problems.

* Corresponding Author. Email: makbay@iiia.csic.es.
[1] Equal contribution.

Construct, Merge, Solve and Adapt (CMSA) [7] is a general-purpose hybrid metaheuristic framework designed to tackle combinatorial optimization problems. The core idea of CMSA is to iteratively construct and solve reduced problem instances—referred to as subinstances—which are composed of solution components extracted from previously generated feasible solutions.

The algorithm applies four main steps iteratively. In the CONSTRUCT phase, a number of solutions are generated probabilistically. The MERGE step then builds a subinstance by combining the components from these generated solutions. Next, the SOLVE step applies an exact solver or method aiming to find an optimal solution to the current subinstance. Finally, the ADAPT step systematically updates the subinstance based on the solution obtained. Specifically, this step removes components that have persisted in the subinstance for a long time but are not present in the solution obtained by the exact solver, thereby refining the search space over time.

The CMSA metaheuristic has been successfully applied to a wide range of combinatorial optimization problems. Early applications focused on the minimum covering arborescence and minimum common string partition problems [7]. More recent studies have extended its use to diverse domains such as test data generation for software product lines [12], the maximum happy vertices problem [21], and the maximum tree coverage problem [22]. Beyond academic benchmarks, CMSA has also demonstrated its effectiveness on real-world problems, including complex scheduling and routing tasks [10, 3, 19, 1].

To further enhance the performance and applicability, several CMSA variants have been introduced. One notable extension is the self-adaptive CMSA, which dynamically adjusts key algorithm parameters and biases solution construction toward the so far best-found solution [2]. This variant has shown strong results on problems such as multi-dimensional multi-way number partitioning [9] and two-echelon electric vehicle routing with simultaneous pickup and deliveries [3]. Another recent contribution is the Multi-Constructor CMSA [20], which integrates multiple solution constructors into the CMSA framework. These constructors are selected based on reinforcement learning-driven probabilities that are updated throughout the search. Applied to the Maximum Disjoint Dominating Sets Problem, this variant achieves state-of-the-art performance across thousands of benchmark instances, particularly excelling on large-scale networks.

A further advancement is RL-CMSA [18], which incorporates a reinforcement learning mechanism to guide the solution construction process, eliminating the need for handcrafted greedy evaluation functions. RL-CMSA improves not only performance across problem domains but also the overall adaptability and learning capabilities of the CMSA framework.

In this work, we propose a novel extension of CMSA by integrating it with Column Generation to solve the Variable-Sized Bin Packing Problem (VSBPP). Our approach retains CMSA's original probabilistic solution construction phase but enhances the search process by including a Column Generation step that iteratively generates solution components, i.e., bin configurations in our case, based on dual values derived from a linear relaxation. As a result, solution components obtained from both, the heuristic constructor and the Column Generation process, are merged into the subinstance. An Integer Programming refinement phase is applied to extract high-quality feasible solutions, followed by an adaptation step that prunes outdated bin configurations from the pool. This integration allows our method to benefit from the complementary strengths of CMSA and Column Generation, balancing diversification through probabilistic construction with intensification through linear programming-guided column generation. Our experimental results show that this hybrid strategy achieves a significant improvement in terms of both the best and average solution quality on nearly all VSBPP benchmark instances, while requiring less computation time than competing approaches.

The remainder of this paper is organized as follows: Section 2 presents the problem definition; Section 3 details our proposed method; Section 4 reports experimental results; and Section 5 concludes the work with final remarks and future research directions.

2 Problem Description

The *VSBPP* is a version of the classical bin packing problem in which bins come in multiple types, each with its own capacity and cost. Formally, we are given a set $S = \{1, \dots, n\}$ of n items, where each item $i \in S$ has a strictly positive weight $w_i > 0$, and a set $B = \{1, \dots, m\}$ of m bin types. Each bin type $k \in B$ is defined by its capacity $W_k > 0$ and cost $C_k > 0$, with the assumption (without loss of generality) that $W_1 < W_2 < \cdots < W_m$. Moreover, we also assume that $C_1 < C_2 < \cdots < C_m$, as we can always remove a bin type having at least the same cost and no more capacity than some other bin type from consideration. The objective is to pack all items into a set of bins such that: (i) each item is placed in exactly one bin, (ii) the total weight of items in each bin does not exceed the capacity of the selected bin type, and (iii) the total cost of the used bins is minimized. Importantly, each bin type can be used an unlimited number of times.

Several approaches have been developed to tackle the VSBPP. In this work, we concentrate on the original version of the problem, where—as noted above—an unlimited number of bins per type is available. Crainic et al. [8] studied a more general variant with explicit limits on the number of bins per type, proposing lower bounds and heuristic methods. Haouari and Serairi [14] introduced various greedy heuristics alongside a genetic algorithm to address the problem. Later, Hemmelmayr et al. [15] proposed an advanced variable neighborhood search (VNS) algorithm, which remains among the most effective methods for the original VSBPP. Since then, most research has shifted toward variants of the VSBPP incorporating additional constraints. Recent examples include the VSBPP with time windows [13] and the VSBPP with conflict constraints [11]. Recently, Akbay et al. [4] investigated different ILP formulations for solving subinstances of the VSBPP within the CMSA framework. They showed that set-covering-based models, compared to assignment-type models, lead to significantly better performance, resulting in new best-known solutions for 68 out of 150 benchmark instances and advancing the state of the art for the original problem.

2.1 Assignment-Type ILP Model

A standard way to model the VSBPP is through an assignment-type integer linear programming (ILP) model [14]. The model uses binary variables x_{ij} and y_{jk} where:

- $x_{ij} = 1$ if item i is assigned to bin j, and 0 otherwise,
- $y_{jk} = 1$ if bin j is of type k, and 0 otherwise.

Assuming an upper bound of n bins (equal to the number of items), the ILP formulation becomes:

$$\min \quad \sum_{j=1}^{n} \sum_{k=1}^{m} C_k \cdot y_{jk} \tag{1}$$

$$\text{s.t.} \quad \sum_{j=1}^{n} x_{ij} = 1 \qquad \forall i \in S \tag{2}$$

$$\sum_{k=1}^{m} y_{jk} \leq 1 \qquad \forall j = 1, \dots, n \tag{3}$$

$$\sum_{i=1}^{n} w_i \cdot x_{ij} \leq \sum_{k=1}^{m} W_k \cdot y_{jk} \qquad \forall j = 1, \dots, n \tag{4}$$

$$x_{ij} \in \{0, 1\}, \qquad \forall i \in S,\ j = 1, \dots, n \tag{5}$$

$$y_{jk} \in \{0, 1\} \qquad \forall j = 1, \dots, n,\ \forall k \in B \tag{6}$$

Equations (2) ensure each item is assigned to exactly one bin. Inequalities (3) guarantee that each bin is assigned at most one bin type, and constraints (4) enforce that the total weight of items in a bin does not exceed its capacity. This model is compact and well-suited for small- to medium-sized instances, but it becomes inefficient for larger ones due to the high number of variables and constraints.

2.2 Set-Covering Based ILP Model

An alternative formulation models the VSBPP using a set-covering framework. Let $\mathcal{B}$ denote the set of all feasible bin configurations, i.e., all non-empty subsets of items whose total weight does not exceed the capacity of some bin type. For each configuration $b \in \mathcal{B}$, let c_b be its cost, defined as the cost of the smallest bin type that can accommodate all items in b. Let $\mathcal{B}_i =\subseteq \mathcal{B}$ be the subset of configurations that contain item i. The set-covering-based ILP formulation can be stated as follows [4]:

$$\min \quad \sum_{b \in \mathcal{B}} c_b \cdot x_b \tag{7}$$

$$\text{s.t.} \quad \sum_{b \in \mathcal{B}_i} x_b \geq 1 \qquad \forall i \in S \tag{8}$$

$$x_b \in \{0, 1\} \qquad \forall b \in \mathcal{B} \tag{9}$$

Here, each binary variable x_b indicates whether configuration b is selected. Inequalities (8) ensure that each item is included in at least one selected bin configuration. Although this formulation introduces

exponentially many variables (one for each possible bin configuration), it provides a stronger liner programming (LP) relaxation and a more scalable foundation for advanced solution techniques such as Column Generation (CG) [5]. Indeed, the set-covering model is often used as the master problem in decomposition approaches for bin packing and related problems.

We remark that a solution obtained by the set-covering model may contain some items more than once. A set partitioning model, in which inequality (8) is replaced by a corresponding equality would avoid this. Nevertheless, the set-covering model is usually preferred as it is a often easier to solve and multiple appearances of some items can finally be eliminated from a solution in a straight-forward way.

Our proposed algorithm builds on this set-covering formulation of the VSBPP and leverages CG as an additional mechanism for dynamically generating promising bin configurations within the CMSA framework. Instead of pre-enumerating all feasible configurations, we iteratively generate new columns, i.e., bin configurations, using dual information from the restricted master problem. This not only addresses the combinatorial explosion of possible configurations but also enhances the intensification capability of CMSA by guiding the search toward cost-effective solutions derived from the LP relaxation.

3 Solution Approach

The proposed hybrid method combines CMSA with CG (CMSA-COLGEN) to solve the VSBPP. While the core CMSA framework is retained for solution construction and diversification, the integration of CG enables a systematic exploration of the solution space through linear relaxations and pricing subproblems. In contrast to previous CMSA implementations (i.e., CMSA-SETCOV algorithm [4]), which relied solely on heuristic bin configuration generation, our approach also incorporates dual information derived from an LP master problem to dynamically identify cost-effective packing patterns.

Following the CMSA paradigm, we first define a complete set of solution components C. In the context of the VSBPP, each valid bin configuration $b \in \mathcal{B}$ corresponds to a solution component c^b. Thus, the complete set of solution components is given by

$$C := \{c^b \mid b \in \mathcal{B}\},$$

where $\mathcal{B}$ denotes the set of all valid bin configurations (i.e., feasible groupings of items respecting bin capacities).

A feasible solution to the VSBPP corresponds to a subset $S \subset C$ such that each item is covered exactly once by some bin configuration in S.

3.1 Overview of the Algorithmic Framework

Algorithm 1 shows CMSA-COLGEN in pseudo-code. In each iteration, a number of solutions—controlled by the parameter n_{a}—are generated using a randomized greedy construction strategy, identical to that employed in CMSA-SETCOV [4]. The bin configurations from these constructed solutions are then merged into a global pool C' (lines 12–14) which serves as the initial state for the current subinstance.

Subsequently, in line 17 the COLUMN GENERATION phase is executed. This phase is initialized using the current subinstance as the restricted master problem in the classical column generation scheme. The global bin pool C' is then leveraged to solve a restricted linear relaxation of the set covering problem. The resulting dual values associated with the item coverage constraints are used to guide

Algorithm 1 CMSA-COLGEN

1: **input:** CMSA parameters: number of solutions per iteration n_{a}, maximum age $age_{\max}$, ILP solve time limit t_{ILP}, column generation LP iteration limit it_{CG}.
2: $S^{\mathrm{bsf}} \leftarrow \emptyset$ {Best solution found so far}
3: $C' \leftarrow \emptyset$ {Pool of solution components ($\hat{=}$ subinstance)}
4: **while** CPU time limit not reached **do**
5: **for** $i \leftarrow 1$ to n_{a} **do**
6: $S \leftarrow$ ProbabilisticSolutionConstruction()
7: **if** $f(S) < f(S^{\mathrm{bsf}})$ **then**
8: $S^{\mathrm{bsf}} \leftarrow S$
9: **end if**
10: **for** each $c \in S$ such that $c \notin C'$ **do**
11: $C' \leftarrow C' \cup \{c\}$
12: $age[c] \leftarrow 0$
13: **end for**
14: **end for**
15: *# Price in up to it_{CG} new columns by CG:*
16: $C' \leftarrow$ ColumnGeneration(C', it_{CG})
17: $S^{\mathrm{ILP}} \leftarrow$ SolveSubinstance(C', t_{ILP})
18: **if** $f(S^{\mathrm{ILP}}) < f(S^{\mathrm{bsf}})$ **then**
19: $S^{\mathrm{bsf}} \leftarrow S^{\mathrm{ILP}}$
20: **end if**
21: Adapt(C', S^{ILP}, $age_{\max}$)
22: **end while**
23: **output:** S^{bsf}

a knapsack-based pricing subproblem. For each bin type, a dynamic programming algorithm is applied to solve the pricing problem and possibly identify new bin configurations (i.e., columns) with negative reduced cost. These columns are incorporated into the model, and this procedure is repeated until no column with a reduced cost below a small threshold $-\epsilon$ is found or until the pre-defined iteration limit it_{CG} is reached.

After the CG phase of the current iteration is completed, the enriched subinstance—now extended with the newly generated columns—is passed to the SOLVE phase. In this phase, the 0-1 integer set covering problem over the bin configuration pool C' is solved to obtain an integer solution for the current subinstance (cf. line 18).

Finally, in the ADAPT phase, the subinstance is updated in line with the traditional CMSA mechanism. Specifically, the age values of all components in $C' \setminus S^{\mathrm{ILP}}$ are incremented by one, while components in S^{ILP} have their age values reset to zero. Components whose age values reach the predefined threshold $age_{\max}$ are removed from C' to avoid bloating the model with rarely used components. The age value $age[c]$ of a solution component $c \in C$ reflects the number of consecutive iterations for which c has been part of the subinstance C' without being included in the ILP solution S^{ILP}.

These steps are repeated iteratively until the overall CPU time limit is reached.

3.1.1 Probabilistic Solution Construction

At each iteration of CMSA-COLGEN, solutions are constructed using the probabilistic solution construction procedure proposed in [4]. In this procedure, each used bin is represented as a subset $b \subseteq S$ of the so-far packed items. To guide the construction process, we evaluate each bin using four key properties.

First, the *load* of a bin $b_{\mathrm{load}} = \sum_{i \in b} w_i$ represents the total weight of the packed items. Next, the *type* of a bin, b_{type}, is determined as

Algorithm 2 Probabilistic construction of a valid VSBPP solution

1: **input:** values for solution construction parameters d_{rate}, l_{size}
2: Let $(i_1, \ldots, i_n)$ be a randomly ordered list of all n items
3: $b^{\text{init}} := \{i_1\}$ *# first bin is initialized with first item*
4: $B := \{b^{\text{init}}\}$
5: **for** $l := 2, \ldots, n$ **do**
6: $O := \emptyset$ *# set of possible placements for item i_l*
7: **for** $b \in B$ **do**
8: **if** $b_{\text{load}} + w_{i_l} \leq \max_{\text{load}}$ **then**
9: $b^+ := b \cup \{i_l\}$
10: $O := O \cup \{b^+\}$
11: **end if**
12: **end for**
13: **if** O is non-empty **then**
14: $b^+ :=$ ChooseOption(O, d_{rate}, l_{size})
15: $B := B \setminus \{b\} \cup \{b^+\}$
16: **else**
17: $b^{\text{new}} := \{i_l\}$ *# new bin containing item i is created*
18: $B := B \cup \{b^{\text{new}}\}$
19: **end if**
20: **end for**
21: Sort(B)
22: $S :=$ ExtractSolutionComponents(B)
23: **output:** S

the one whose capacity $W_{b_{\text{type}}}$ can accommodate the load and whose cost are smallest, i.e., $b_{\text{type}} := \min\{k \in K \mid W_k \geq b_{\text{load}}\}$. Based on the assigned type, the *cost* of a bin is given by $b_{\text{cost}} = C_{b_{\text{type}}}$, corresponding to the cost of the selected bin type. Additionally, we define the *cost-efficiency* of a bin as the ratio $b_{\text{ratio}} = b_{\text{cost}}/b_{\text{load}}$, which reflects how efficiently the bin's cost scales with its load. Lastly, we define as an upper bound for any bin's load $\max_{\text{load}} := \max\{W_j \mid j \in K\}$, i.e., the maximum over all bin types' capacities.

The full procedure for the probabilistic solution construction is outlined in Algorithm 2. The algorithm begins by randomly shuffling the items into an order $(i_1, \ldots, i_n)$, as shown in line 3. The construction is then initialized by placing the first item into a new bin, whose metrics (load, type, cost, and ratio) are computed accordingly. Then, for each subsequent item, the procedure evaluates possible placements across the current set of bins. Among these options, preference is probabilistically given to one that yields the lowest cost-efficiency ratio, as lower ratios in general indicate more desirable placements. This selection mechanism is implemented in function call ChooseOption(O, d_{rate}, l_{size}) (line 14), where O denotes the current set of placement options.

The selection applied within this function proceeds as follows. With probability d_{rate}, an option with lowest ratio is selected. Otherwise, the function considers the $\max\{|O|, l_{\text{size}}\}$ options with the best (i.e., lowest) ratios and selects one of these uniformly at random, thus introducing controlled stochasticity into the construction process. Once all items have been placed, the resulting bins are sorted in ascending order of their cost-efficiency ratio by Sort(B) (line 21). As a tie-breaking rule, bins with smaller minimum item indices are preferred.

Finally, the solution is encoded as a set of solution components via the ExtractSolutionComponents(B) call (line 22). In this step, for each bin $b \in B$, the corresponding solution component c^b is simply added to the constructed solution set S.

3.2 Utilizing Column Generation in CMSA

CG is a well-established decomposition technique in mathematical programming, particularly effective for large-scale combinatorial optimization problems such as routing, scheduling, and packing [5]. By solving an LP relaxation over a restricted set of columns (bin configurations) and iteratively generating new columns through solving a pricing subproblem guided by dual values, CG may drastically improve the quality of CMSA subinstance pools. This integration strengthens the CMSA framework by adding an exact subproblem decomposition perspective to its existing metaheuristic base.

The CG phase begins by solving the LP relaxation of the set-covering model using the current pool of bin configurations. The restricted master problem is formulated as:

$$\min_{\lambda} \quad \sum_{b \in \mathcal{B}^{\text{RMP}}} c_b \cdot \lambda_b \tag{10}$$

$$\text{s.t.} \quad \sum_{p \in \mathcal{B}^{\text{RMP}}} a_{ib} \cdot \lambda_b \geq 1 \qquad \forall i \in S \tag{11}$$

$$\lambda_b \geq 0 \qquad \forall b \in \mathcal{B}^{\text{RMP}} \tag{12}$$

Here, $\mathcal{B}^{\text{RMP}} \subseteq \mathcal{B}$ is the current set of bin configurations in the restricted master problem, c_b is the cost of bin configuration b, and $a_{ib} \in \{0, 1\}$ indicates with value one if item i is packed in bin configuration b.

After solving the LP, dual variable values of constraints π_i are extracted for each of the constraints (11). We are now looking for some bin configurations $b \in \mathcal{B}$ with negative reduced cost $\bar{c}_b = C_k - \sum_i \pi_i z_i$, as such configurations can potentially improve the objective value. To find such bin configurations, we solve the pricing subproblem, which breaks down into a 0–1 knapsack problem for each bin type $k \in B$:

$$\max_{z} \quad \sum_{i \in S} \pi_i \cdot z_i \tag{13}$$

$$\text{s.t.} \quad \sum_{i \in S} w_i \cdot z_i \leq W_k \tag{14}$$

$$z_i \in \{0, 1\} \quad \forall i \in S \tag{15}$$

Here, variables z_i indicate the inclusion of item i in a the configuration, and the objective is to maximize the total dual value of selected items, which corresponds to the largest potential saving in cost. If a bin configuration with total dual value larger than C_k is obtained, it can be helpful and is therefore added to the bin pool of CMSA.

To efficiently solve the knapsack subproblems, classical dynamic programming is employed. We define the state $f[s]$ as the maximum dual profit that can be attained with a total weight s, for $0 \leq s \leq C$, and update a dynamic programming table using the recurrence

$$f[s] = \max\{f[s],\ p_j + f[s - w_j]\} \quad \text{for all } s \geq w_j.$$

A traceback procedure is then applied to recover the selected subset of items corresponding to the optimal profit.

3.3 Integer Programming Refinement and Pool Adaptation

After the LP phase, the algorithm optionally performs an integer programming refinement using the full bin pool as input to the set-covering model. This phase phase may yield an improved solution

by selecting a cost-minimizing subset of bins. A post-processing reduction removes redundant or suboptimal bins that over-cover items.

The final *Adapt* step incrementally ages bins that have not been used in recent high-quality solutions. Bins exceeding a maximum age threshold are pruned from the pool, helping to control memory usage and promote search diversification in subsequent iterations.

4 Experimental Evaluation

The proposed algorithm was implemented in C++ and the experiments were conducted using a cluster of machines equipped with Intel® Xeon® 5670 CPUs having 12 cores of 2.933 GHz and at least 32 GB of RAM. To solve the respective subinstances within both versions of CMSA, we used CPLEX 22.1 in single-threaded mode.

In addition to the CMSA variants with and without CG we also consider a CG-standalone algorithm. In this version, CG is initialized with a solution generated using the same solution construction method as in the CMSA algorithm. CG is then performed until no further columns with negative reduced cost can be found, thereby solving the LP relaxation to optimality. Afterwards, the corresponding reduced integer master problem is solved once on top of it. While no time limit is applied during the CG phase, a time limit of 150 seconds is imposed for solving the ILP model, which is consistent with the time limit used for the CMSA variants.

Problem Instances. The set of problem instances used in this study was introduced in [14]. This benchmark set comprises seven bin types with capacities defined as $W_1 = 70$, $W_2 = 100$, $W_3 = 130$, $W_4 = 160$, $W_5 = 190$, $W_6 = 220$, and $W_7 = 250$. Item weights are randomly selected integers from the interval $[1, 250]$. The benchmark set is organized into three different classes of instances. Specifically, class B1 employs a linear bin cost function, $C_i = W_i$ for $i = 1, \ldots, 7$, class B2 utilizes a concave cost function, $C_i = \lceil 10\sqrt{W_i} \rceil$, and class B3 applies a convex cost function, $C_i = \lceil 0.1, W_i^{3/2} \rceil$, with $i = 1, \ldots, 7$. For each combination of the number of items, $n \in 100, 200, 500, 1000, 2000$, and bin cost function class, 10 problem instances were generated, resulting in a total of 150 instances. Optimal solutions for these instances remain unknown.

Parameter Tuning. Both CMSA-SETCOV and CMSA-COLGEN require a reasonable parametrization. A tuning process was carried out using the irace tool [16], with each algorithm variant tuned independently for each problem class (B1, B2, B3). A budget of 2000 runs was allocated for each tuning session. For the CMSA-SETCOV algorithm, each run was capped at 150 CPU seconds, in line with [15]. For CMSA-COLGEN, the LP relaxation within the CG phase was subject to a separate iteration limit parameter, denoted as it_{CG}. The final parameter configurations used in the following experiments are shown in Table 1.

Numerical Results. We compare CMSA-COLGEN with two current state-of-the-art techniques: the VNS from [15] and the CMSA-SETCOV approach from [4], plus the standalone CG method. Comparative results are presented in the tables and the pairwise stacked-bar advantage plots (Figure 1). For brevity, the detailed results shown in the tables include only the best-known outcomes along with key performance measures, and—due to space limitations—the complete detailed results are reported for the B3 instance class. Numerical results for instance classes B1 and B2 are summarized in the text and in the stacked-bar plots. In the table, the first two columns indicate the instance characteristics (the number of items, n, and the instance number). The next column presents the best-known solution value. This is followed by three multi-columns reporting the results for standalone CG, CMSA-SETCOV, and CMSA-COLGEN. For the standalone CG method, these multi-columns show the LP lower bound, the IP solution value obtained after solving the set-covering model on the collected bins, the total computational time (which includes both the CG phase and the subsequent IP solve), and the optimality gap of the IP solution. For the CMSA-SETCOV and CMSA-COLGEN approaches, the columns labeled "best", "avg", and "avg. time" indicate, respectively, the best solution value found over ten runs, the average final solution value, and the average time at which the final solutions were achieved (within a 150 CPU-second time limit).

Overall, CMSA-COLGEN demonstrates a significant improvement over the other methods, with its superiority being more pronounced for the B3 instance class and for larger problem sizes (notably when $n = 1000$ or 2000). In terms solution quality, the pairwise comparisons indicate that CMSA-COLGEN outperforms both the standalone CG and CMSA-SETCOVapproaches in both best and average performance. Moreover, these improvements are achieved with shorter average computational times. With the exception of a single instance where CMSA-COLGEN produced a solution marginally worse than the current best-known result, CMSA-COLGEN consistently delivered solutions that are equal to or better than the best-known solution values. In fact, CMSA-COLGEN has found three new best-known solutions for B1 instances, five for B2, and twelve for B3.

In addition to the detailed result table, Figure 1 presents a comparative summary of the algorithms' performance in the form of *pairwise stacked-bar advantage plots*. The top row (panels a–c) shows results for the best objective values, while the bottom row (panels d–f) shows the corresponding plots for the average objective values. Within each row the three panels correspond to instance classes B1, B2, and B3, respectively. Each panel is further subdivided into cells that compare algorithms *pairwise*: the row corresponds to one algorithm and the column to another.

Within each cell, the stacked bar is split into three colored segments. Green indicates the number of instances on which the row algorithm achieved a strictly *better* result (lower solution value) than the column algorithm. Gray indicates the number of instances for which the two algorithms *tied*, i.e., obtained the same result. Red indicates the number of instances on which the row algorithm performed *worse*. Hence, each cell summarizes how often the row algorithm outperforms, equals, or underperforms the column algorithm across all instances of that panel. As these figures confirm, CMSA-COLGEN generally provides better (or at least equally good) best and average solutions, and it identifies numerous new best-known results.

To statistically validate the observed performance differences among the algorithms, we conducted Wilcoxon signed-rank tests on both best and average performance values across all instances and within relevant subsets (i.e., each instance class and each item size category). The results are summarized in Table 3, with additional breakdowns provided in Table 4. Across all comparisons, the results obtained by CMSA-COLGEN are statistically superior to those of both the standalone column generation (CGIP) and the CMSA-SETCOV baseline. Notably, in both best and average performance, CMSA-COLGEN consistently achieves statistically significant improvements (Holm-adjusted p-values < 0.05) in all global and subset tests except for a few cases involving small instance sizes (e.g., $n = 100$ and $n = 200$), where the differences were less pronounced. These tests confirm that the improvements observed in solution quality and computational efficiency are statistically significant.

To further analyze the role of the CG in the performance of CMSA-COLGEN, we also examine the proportion of bins actually contributed by the heuristic solution generation and the CG phase

Table 1: Parameter settings for CMSA-SETCOV and CMSA-COLGEN for the three instance classes obtained from tuning.

Parameter	Domain	Class B1		Class B2		Class B3	
		CMSA-SETCOV	CMSA-COLGEN	CMSA-SETCOV	CMSA-COLGEN	CMSA-SETCOV	CMSA-COLGEN
t_{ILP}	$(0.3, 30.0)$	15.21	19.54	23.12	21.38	28.07	28.94
l_{size}	$[1, 10]$	10	9	5	3	8	3
d_{rate}	$(0.0, 0.99)$	0.70	0.35	0.64	0.41	0.97	0.43
n_{a}	$[2, 50]$	13	6	50	22	46	49
$age_{\max}$	$[1, 10]$	3	4	1	1	1	8
it_{CG}	$[100, 5000]$	–	2000	–	1500	–	500

Table 2: Results for instances of class B3 (linear cost function).

		best	CGStandalone				CMSA-SETCOV			CMSA-COLGEN		
n	#	known	LpLB	IP	time	Gap	best	avg	avg. time	best	avg	avg. time
100	1	19364	19344.50	**19364**	0.030	0.10	**19364**	19364.00	0.020	**19364**	19364.00	0.030
100	2	19000	18998.67	**19000**	0.030	0.01	**19000**	19000.00	0.030	**19000**	19000.00	0.030
100	3	18272	18272.00	**18272**	0.020	0.00	**18272**	18272.00	0.010	**18272**	18272.00	0.020
100	4	19016	19016.00	**19016**	0.030	0.00	**19016**	19016.00	0.020	**19016**	19016.00	0.020
100	5	16612	16600.50	16622	0.040	0.13	**16612**	16612.00	0.040	**16612**	16612.00	0.040
100	6	18632	18620.50	18640	0.030	0.10	**18632**	18632.00	0.060	**18632**	18632.00	0.030
100	7	18682	18670.50	18690	0.020	0.10	**18682**	18682.00	0.040	**18682**	18682.00	0.030
100	8	19517	19517.00	**19517**	0.030	0.00	**19517**	19517.00	0.020	**19517**	19517.00	0.020
100	9	17950	17950.00	**17950**	0.030	0.00	**17950**	17950.00	0.030	**17950**	17950.00	0.020
100	10	17127	17114.17	17140	0.040	0.15	**17127**	17127.00	0.020	**17127**	17127.00	0.030
200	1	35423	35411.50	35428	0.130	0.05	**35423**	35423.00	0.790	**35423**	35423.00	0.100
200	2	36362	36362.00	**36362**	0.110	0.00	**36362**	36362.00	6.800	**36362**	36362.00	0.080
200	3	33390	33389.00	**33390**	0.140	0.00	**33390**	33390.00	0.300	**33390**	33390.00	0.150
200	4	34327	34314.50	**34327**	0.120	0.04	**34327**	34327.00	0.130	**34327**	34327.00	0.120
200	5	38055	38055.00	**38055**	0.080	0.00	**38055**	38055.00	0.090	**38055**	38055.00	0.060
200	6	35009	35009.00	35032	0.070	0.07	**35009**	35009.00	0.100	**35009**	35009.00	0.070
200	7	38175	38162.17	38183	0.080	0.05	**38175**	38175.00	0.470	**38175**	38175.00	0.110
200	8	36003	36003.00	**36003**	0.070	0.00	**36003**	36003.00	0.090	**36003**	36003.00	0.060
200	9	32700	32700.00	**32700**	0.100	0.00	**32700**	32700.00	1.750	**32700**	32700.00	0.080
200	10	36998	36986.50	**36998**	0.070	0.03	**36998**	36998.00	0.180	**36998**	36998.00	0.080
500	1	94768	94768.00	**94768**	0.490	0.00	**94768**	94768.00	1.110	**94768**	94768.00	0.380
500	2	97983	97983.00	**97983**	0.410	0.00	**97983**	97983.00	0.840	**97983**	97983.00	0.320
500	3	95832	95832.00	**95832**	0.530	0.00	**95832**	95832.00	1.220	**95832**	95832.00	0.380
500	4	91068	91067.00	**91068**	0.440	0.00	**91068**	91068.00	0.450	**91068**	91068.00	0.430
500	5	87676	87664.50	**87676**	0.660	0.01	**87676**	87676.00	0.630	**87676**	87676.00	0.570
500	6	83124	83112.50	83134	0.650	0.03	**83124**	83124.00	21.240	**83124**	83124.00	0.500
500	7	90407	90395.50	**90407**	0.540	0.01	**90407**	90407.00	0.970	**90407**	90407.00	0.430
500	8	87059	87059.00	**87059**	0.730	0.00	**87059**	87059.00	7.370	**87059**	87059.00	0.460
500	9	87398	87398.00	**87398**	0.630	0.00	**87398**	87398.00	2.500	**87398**	87398.00	0.480
500	10	90541	90529.50	90582	0.660	0.06	90543	90543.00	23.880	**90541**	90541.00	0.460
1000	1	176950	176936.50	176956	1.990	0.01	176953	176955.30	98.430	**176948**	176948.00	2.410
1000	2	180989	180975.50	181028	1.940	0.03	180989	180990.60	73.800	**180987**	180987.00	2.760
1000	3	182754	182741.50	182762	2.560	0.01	**182754**	182754.60	58.440	**182754**	182754.00	2.300
1000	4	180857	180857.00	**180857**	2.260	0.00	**180857**	180859.30	58.480	**180857**	180857.00	3.360
1000	5	179154	179142.50	179185	2.050	0.02	**179154**	179154.80	61.790	**179154**	179154.00	5.260
1000	6	188838	188824.50	**188836**	1.800	0.01	188839	188840.20	34.810	**188836**	188836.00	1.920
1000	7	178183	178181.00	**178181**	2.430	0.00	178183	178189.70	89.800	**178181**	178181.00	2.340
1000	8	177459	177447.50	**177459**	2.340	0.01	**177459**	177459.00	14.700	**177459**	177459.00	2.260
1000	9	181001	181001.00	181032	2.160	0.02	**181001**	181007.70	69.930	**181001**	181001.00	2.270
1000	10	176902	176902.00	176933	2.060	0.02	**176902**	176917.20	49.650	**176902**	176902.00	3.270
2000	1	356235	356212.00	356279	10.840	0.02	356235	356236.60	85.250	**356212**	356214.30	32.160
2000	2	369811	369799.50	369829	9.180	0.01	**369811**	369831.70	68.470	**369811**	369811.00	9.700
2000	3	364527	364501.50	364531	9.050	0.01	364527	364562.30	72.100	**364513**	364513.00	10.620
2000	4	356958	356946.50	357032	10.220	0.02	**356958**	356969.80	102.760	**356958**	356958.00	15.820
2000	5	365557	365515.50	365545	10.110	0.01	365564	365576.40	106.840	**365527**	365527.00	17.130
2000	6	365116	365098.50	365123	8.780	0.01	365116	365120.80	56.370	**365110**	365110.00	21.390
2000	7	360816	360780.50	360797	9.710	0.00	360816	360838.20	90.210	**360792**	360792.00	11.910
2000	8	371779	371767.00	371774	8.370	0.00	371779	371798.60	96.170	**371767**	371769.30	13.510
2000	9	355723	355678.83	**355691**	9.870	0.00	355726	355745.30	104.490	**355691**	355691.00	21.920
2000	10	357036	357018.50	357079	8.790	0.02	**357036**	357040.10	68.100	**357030**	357030.00	13.280

Table 3: Wilcoxon signed-rank test results comparing algorithm pairs across all instances (Best & Average Performance).

Metric	Algorithm A	Algorithm B	Statistic	p_{raw}	Median (A−B)	Mean % diff	p_{holm}
Best	CMSA-COLGEN	CGStandalone	0.0	1.60×10^{-18}	-10.0	-0.0306	6.42×10^{-18}
	CMSA-SETCOV	CGStandalone	240.0	7.10×10^{-16}	-10.0	-0.0298	2.13×10^{-15}
	CMSA-COLGEN	CMSA-SETCOV	1.5	1.08×10^{-5}	0.0	-0.0009	2.17×10^{-5}
	CGStandalone	CMSA-SETCOV	5013.0	1.00	10.0	0.0298	1.00
Average	CMSA-COLGEN	CGStandalone	0.0	1.64×10^{-18}	-10.0	-0.0300	6.56×10^{-18}
	CMSA-SETCOV	CGStandalone	604.5	5.88×10^{-13}	-7.70	-0.0283	1.76×10^{-12}
	CMSA-COLGEN	CMSA-SETCOV	41.0	2.76×10^{-9}	0.0	-0.0017	5.53×10^{-9}
	CGStandalone	CMSA-SETCOV	5173.5	1.00	7.70	0.0283	1.00

Table 4: Statistical significance of observed differences ($p_{\text{holm}} < 0.05$) for best and average solution values.

Metric	Algorithm A	Algorithm B	all	B1	B2	B3	n=100	n=200	n=500	n=1000	n=2000
Best	CMSA-COLGEN	CGStandalone	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
	CMSA-SETCOV	CGStandalone	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
	CMSA-COLGEN	CMSA-SETCOV	Yes	No	Yes	Yes	No	No	No	Yes	Yes
Average	CMSA-COLGEN	CGStandalone	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
	CMSA-SETCOV	CGStandalone	Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes
	CMSA-COLGEN	CMSA-SETCOV	Yes	Yes	Yes	Yes	No	No	Yes	Yes	Yes

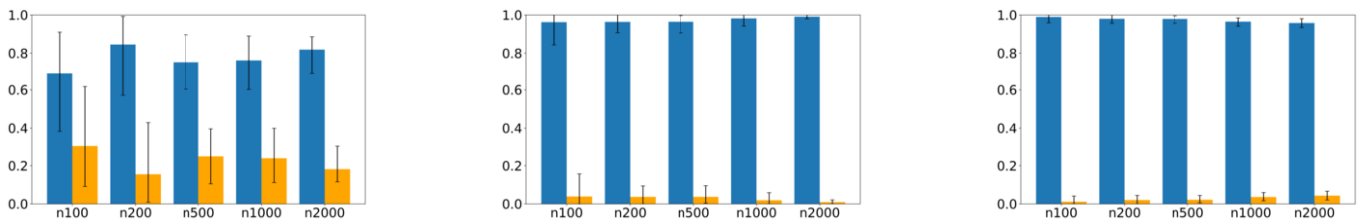
in respect to the final solutions for each instance class. Figure 2 presents the average proportion of bins in the final solution that were introduced specifically by the heuristic solution construction and CG across the three instance classes B1, B2, and B3, together



(a) B1 Instances (b) B2 Instances (c) B3 Instances

(d) B1 Instances (e) B2 Instances (f) B3 Instances

Figure 1: Pairwise stacked-bar advantage plots for **best** solutions (top row, panels a–c) and **average** solutions (bottom row, panels d–f) on instance classes B1, B2, and B3. Each cell reports how often the algorithm named by the row outperforms, ties, or is outperformed by the algorithm named by the column. *Algorithm codes:* BN=Best-Known, VNS=VNS, CGStand=CGStandalone, CMSA-SC=CMSA-SetCov, CMSA-CG=CMSA-ColGen.



(a) B1 Instances (b) B2 Instances (c) B3 Instances

Figure 2: Average proportions of bin patterns in final solutions provided by the construction heuristic (blue bars) and by CG (yellow bars) for instance groups B1, B2, and B3.

with the corresponding variability (error bars). Notably, for B1 we observe both a higher average proportion of CG-added bins and a larger spread, suggesting that the interplay between heuristic bin creation and the bins generated via dual information can fluctuate more in these linear-cost instances. In other words, more bins ultimately forming part of the best solution may come from either source (heuristic or CG) compared to other instances, with that balance varying substantially across different B1 problem instances. By contrast, although B3 exhibits a lower average proportion of CG-added bins than B1, these bins typically prove critical, often including items or patterns that are indispensable to achieving new best-known solutions. Consequently, even a smaller fraction of column-generation-derived bins can make a decisive difference in final solution quality for the B3 instances. Overall, these observations show that the interplay between heuristic construction and CG varies considerably across instance classes. For example, while B1 commonly relies on a larger proportion of CG-generated bins, B3 benefits from fewer but strategically critical columns, often leading to superior solutions.

5 Conclusion

We presented a hybrid approach that combines the CMSA metaheuristic with Column Generation to address the Variable-Sized Bin Packing Problem. Our method leverages the strengths of probabilistic solution construction inherent in CMSA and the powerful LP-guided column generation process. By dynamically generating cost-effective bin configurations via solving knapsack-based pricing subproblems and integrating them into a reduced set-covering model, our hybrid algorithm (CMSA-COLGEN) achieves an excellent balance between diversification and intensification.

The experimental evaluation on a comprehensive benchmark set with linear, concave, and convex cost functions demonstrates that the proposed approach consistently outperforms state-of-the-art techniques such as standalone Column Generation, CMSA-SetCov, and a Variable Neighborhood Search framework. In particular, the new method exhibits superior best-solution quality, enhanced robustness, and reduced computational times, especially on larger instances and on problem classes where the cost functions impose additional challenges. The performance improvements achieved by the proposed hybrid method are consistent and statistically significant, as confirmed by Wilcoxon signed-rank tests below 0.05 across nearly all comparisons.

Future work should explore the extension of the hybrid framework to other challenging optimization domains. More generally, the approach appears highly promising for combinatorial optimization problems that (a) can be reformulated as set covering problems, (b) for which a well working construction heuristic is known or can be developed, and (c) where classical column generation and/or branch-and-price are in principle promising but do not scale well enough the relevant instance sizes.

Acknowledgements

This work was supported by grant PID2022-136787NB-I00 funded by MCIN/AEI/10.13039/501100011033 and by the project Cátedra ENIA UAB-Cruïlla (TSI-100929-2023-2).

References

[1] M. A. Akbay and C. Blum. Application of CMSA to the minimum positive influence dominating set problem. In *Artificial Intelligence Research and Development*, pages 17–26. IOS Press, 2021.

[2] M. A. Akbay, A. López Serrano, and C. Blum. A self-adaptive variant of CMSA: Application to the minimum positive influence dominating set problem. *International Journal of Computational Intelligence Systems*, 15(1):1–13, 2022.

[3] M. A. Akbay, C. B. Kalayci, and C. Blum. Application of adapt-cmsa to the two-echelon electric vehicle routing problem with simultaneous pickup and deliveries. In *European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar)*, pages 16–33. Springer, 2023.

[4] M. A. Akbay, C. Blum, and C. B. Kalayci. Cmsa based on set covering models for packing and routing problems. *Annals of Operations Research*, 343(1):1–38, 2024.

[5] C. Barnhart, E. L. Johnson, G. L. Nemhauser, M. W. Savelsbergh, and P. H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.

[6] C. Blum and A. Roli. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM computing surveys (CSUR)*, 35(3):268–308, 2003.

[7] C. Blum, P. Pinacho Davidson, M. López-Ibáñez, and J. A. Lozano. Construct, merge, solve & adapt: A new general algorithm for combinatorial optimization. *Computers & Operations Research*, 68:75–88, 2016.

[8] T. G. Crainic, G. Perboli, W. Rei, and R. Tadei. Efficient lower bounds and heuristics for the variable cost and size bin packing problem. *Computers & Operations Research*, 38(11):1474–1482, 2011.

[9] M. Djukanović, A. Kartelj, and C. Blum. Self-adaptive CMSA for solving the multidimensional multi-way number partitioning problem. *Expert Systems with Applications*, 232:120762, 2023.

[10] N. Dupin and E.-G. Talbi. Matheuristics to optimize refueling and maintenance planning of nuclear power plants. *Journal of Heuristics*, 27 (1-2):63–105, 2021.

[11] A. Ekici. A large neighborhood search algorithm and lower bounds for the variable-sized bin packing problem with conflicts. *European Journal of Operational Research*, 308(3):1007–1020, 2023.

[12] J. Ferrer, F. Chicano, and J. A. Ortega-Toro. CMSA algorithm for solving the prioritized pairwise test data generation problem in software product lines. *Journal of Heuristics*, 27:229–249, 2021.

[13] K. Fleszar. A new MILP model and fast heuristics for the variable-sized bin packing problem with time windows. *Computers & Industrial Engineering*, 175:108849, 2023.

[14] M. Haouari and M. Serairi. Heuristics for the variable sized bin-packing problem. *Computers & Operations Research*, 36(10):2877–2884, 2009.

[15] V. Hemmelmayr, V. Schmid, and C. Blum. Variable neighbourhood search for the variable sized bin packing problem. *Computers & Operations Research*, 39(5):1097–1108, 2012.

[16] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, and T. Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43 – 58, 2016.

[17] C. H. Papadimitriou and K. Steiglitz. *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998.

[18] J. Reixach and C. Blum. How to improve “construct, merge, solve and adapt"? use reinforcement learning! *Annals of Operations Research*, pages 1–32, 2024.

[19] R. M. Rosati, L. Kletzander, C. Blum, N. Musliu, and A. Schaerf. Construct, merge, solve and adapt applied to a bus driver scheduling problem with complex break constraints. In *International conference of the Italian association for artificial intelligence*, pages 254–267. Springer, 2022.

[20] R. M. Rosati, S. Bouamama, and C. Blum. Multi-constructor cmsa for the maximum disjoint dominating sets problem. *Computers & Operations Research*, 161:106450, 2024.

[21] D. Thiruvady and R. Lewis. Recombinative approaches for the maximum happy vertices problem. *Swarm and Evolutionary Computation*, 75:101188, 2022.

[22] J. Zhou and P. Zhang. Simple heuristics for the rooted max tree coverage problem. In W. Wu and J. Guo, editors, *Combinatorial Optimization and Applications*, pages 252–264, Cham, 2024. Springer Nature Switzerland.